Design and FPGA Implementation of a Scalable SPI Bus Interface Supporting Multiple Sensors with Variable Data Widths

Comprehensive Project Report & Hardware Verification Presentation

Bhargav Vemparala
and
Katta Sri Kaushik

Department of Electrical and Computer Engineering
University Evaluation Board

August 2026

Presentation Outline

1. Introduction & Evolution
2. System Architecture
3. Implemented Methodologies
4. Verilog Implementation
5. FPGA & Web Validation
6. Hardware Signal Capture & Verification
7. Simulation & Case Studies
8. Experimental Results
9. Conclusion & Future Work

Introduction & Motivation

- **Problem with Traditional SPI:**
  - Standard SPI requires a dedicated Chip Select (CS) line for each slave device.
  - For N devices, physical wiring complexity is $O(N)$ (requires $N + 3$ wires).
  - Causes PCB routing congestion and depletes GPIO pins on the host MCU.
- **Proposed Solution:**
  - Architect an intelligent **SPI Bus Interface** on a Xilinx Artix-7 FPGA.
  - Share a single 3-wire/4-wire bus among all heterogeneous sensors.
  - Virtualize sensor registers on-chip, reducing wiring complexity to a constant $O(1)$.
- **Key Innovations:**
  - Dual protocol support: **Address + Bit-Length** (Dynamic) vs. **ROM Lookup Table** (Static).
  - Integrated Clock Domain Crossing (CDC) asynchronous FIFO.
  - Full hardware verification via ESP32 Web Interface and Vivado synthesis.

Design Evolution Across Three Iterations

Iteration 1: PoC

Single Slave Direct

- Point-to-point interface.
- Fixed 1-bit input/output.
- 5-state machine.
- High protocol overhead (5 cycles per bit).

Iteration 2: Burst-Mode

Fixed-Width Bus

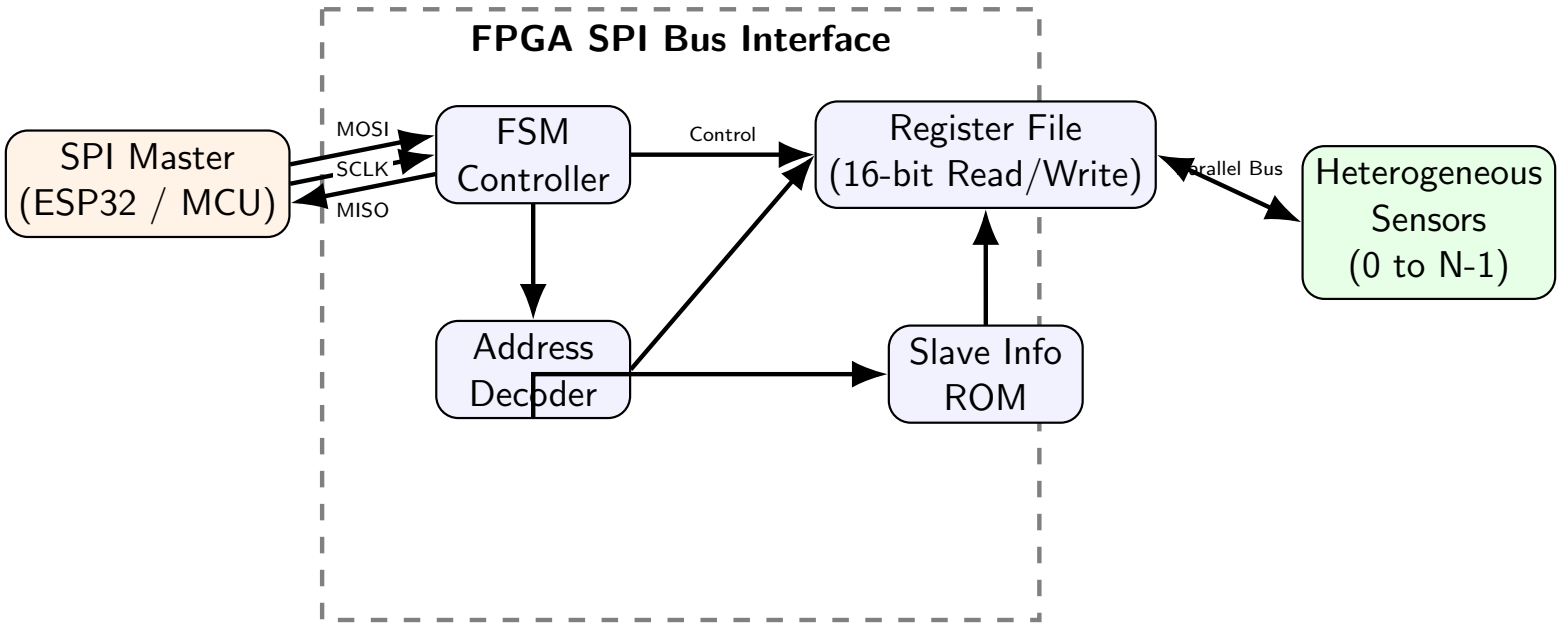
- Address decoding added.
- Fixed 4-bit burst transfers.
- Shared register array.
- Inflexible for asymmetric sensors.

Iteration 3: Scalable Architecture

Final Architecture

- Heterogeneous sensors.
- Asymmetric read/write widths.
- Dynamic & ROM protocols.
- Decoupled master/sensor domains.

Generalized System Block Diagram



Core FPGA Internal Submodules

- **FSM Controller:** Coordinates incoming serial streams on MOSI, decodes command opcodes, and drives serial bit shifting.
- **Address Decoder:** Maps incoming address frames directly to target virtual sensor register slices.
- **Register File:** 16-bit split register space:
  - `read_reg` (16-bit): Captures live parallel sensor readings.
  - `write_reg` (16-bit): Stores master control outputs.
- **Snapshot Controller:** Latches sensor register values into shadow registers at the start of read transactions to prevent data tearing during multi-bit transfers.

Clock Domain Crossing & FIFO Synchronization

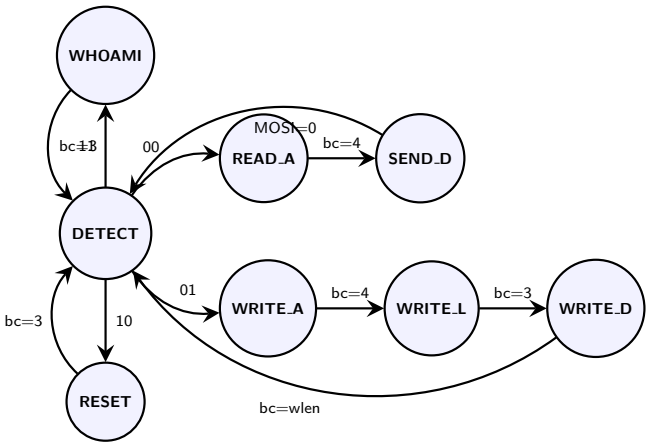
- **Asynchronous Domain Challenge:** Sensors operate on CLK_{sensor} while the SPI host communicates on an independent, bursty SCLK.
- **Metastability Mitigation:**
  - Direct cross-domain sampling leads to timing violations and glitches.
  - Integrated a 16-bit wide, depth-8 **Asynchronous CDC FIFO** buffer.
  - Write port driven by CLK_{sensor}; read port driven by SCLK.
  - Dual-rank Gray-code synchronizers protect read/write pointer boundaries.

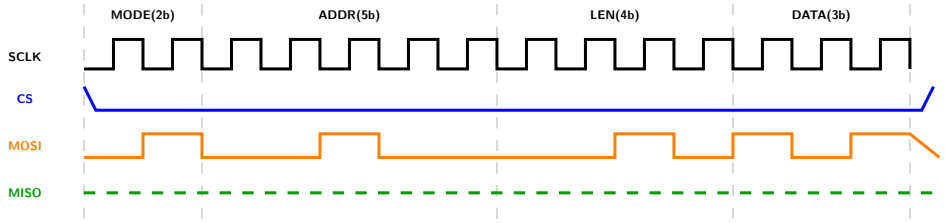
Method 1: Address + Bit-Length Protocol

- **Dynamic Packet Concept:** Master explicitly specifies transaction payload size on the fly for write operations.
- **Packet Breakdown:**
  - **Mode (2 bits):** 00=Read, 01=Write, 10=Reset, 11=WHOAMI.
  - **Address (5 bits):** Target base offset ($A_4 \ldots A_0$).
  - **Length (4 bits):** Transmitted in writes; defines payload bit count minus 1.
- **Master-Terminated Reads:** Reads omit the length field. Master keeps MOSI=1 to continue reading, pulls MOSI=0 to terminate.

Op	Mode (2b)	Address (5b)	Length (4b)	Data Payload
Read	00	$A_4 \ldots A_0$	N/A	Dynamic (MOSI=0 to stop)
Write	01	$A_4 \ldots A_0$	$L_3 \ldots L_0$	Variable ($L+1$ bits)

Method 1: FSM State Diagram & Write Timing





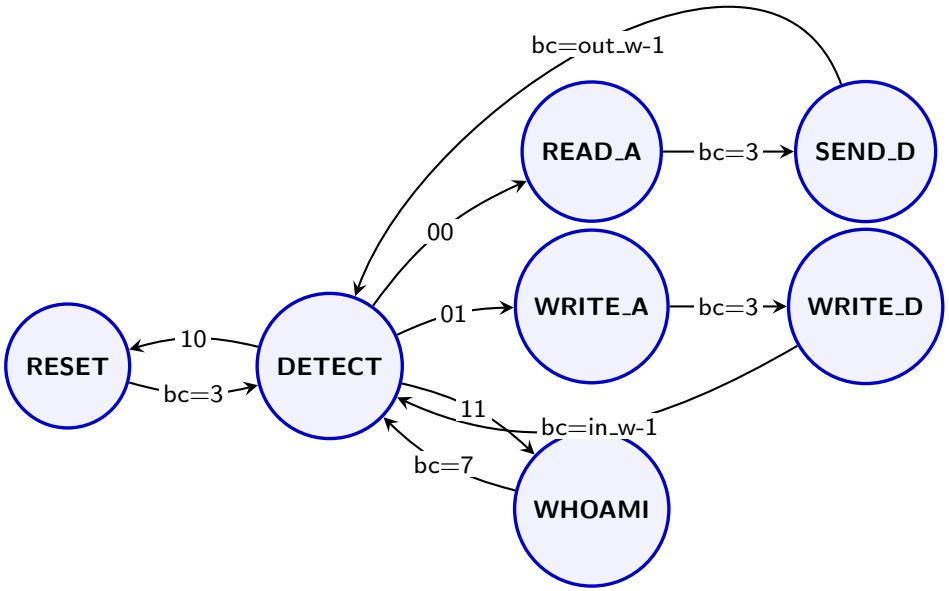
Timing diagram for Write 3 bits (101) to address 0x04.

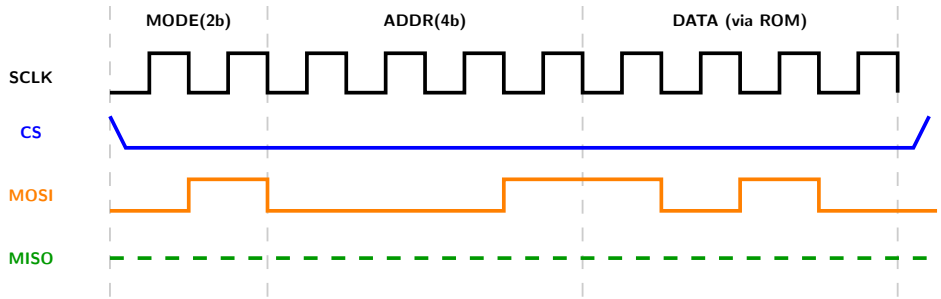
Method 2: ROM Lookup Table Protocol

- **Zero Length Field Overhead:** Sensor register layout metadata is pre-programmed in an on-chip combinational ROM.
- **ROM Table Fields:**
  - `in_width` / `out_width`: Payload bit-widths.
  - `in_start` / `out_start`: Starting bit indices within 16-bit register file.

Address (4b)	In Width	Out Width	In Start	Out Start
`4'd0`	3	3	0	0
`4'd1`	3	3	3	3
`4'd2`	3	3	6	6
`4'd3`	3	3	9	9
`4'd5`	4	4	12	12
Default	1	1	0	0

Method 2: FSM Diagram & Write Timing





Method 2 Write timing (Addr `4'd1`, 4 payload bits from ROM).

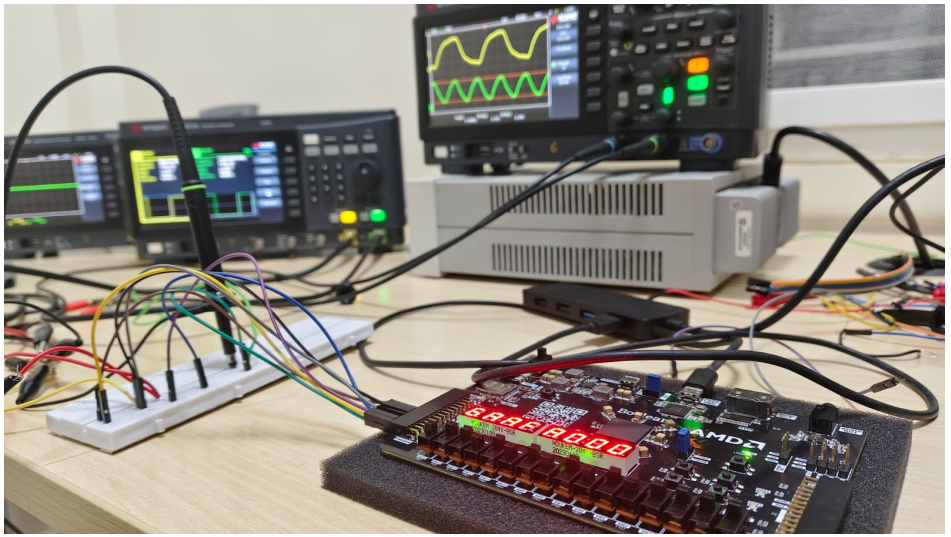
Verilog Design & Synthesis Optimizations

- **Replacement of Modulo Operators:**
  - Modulo operations (`% 16`) synthesize to bulky, high-latency division blocks.
  - Optimized to bitwise AND masks (`& 4'b1111`) for instantaneous 1-cycle logic paths.
- **Single-Driver Assurance:**
  - All register array updates strictly contained within a single synchronous posedge block.
  - Eliminates multi-source driver synthesis conflicts.
- **Tri-state MISO Buffering:**
  - `assign MiSo = (cs == 1'b0 && mode == 2'b00) ? miso_reg : 1'bz;`
  - High-impedance state when `cs` is inactive guarantees physical line sharing.

Physical FPGA Testbench Setup

Hardware Environment:

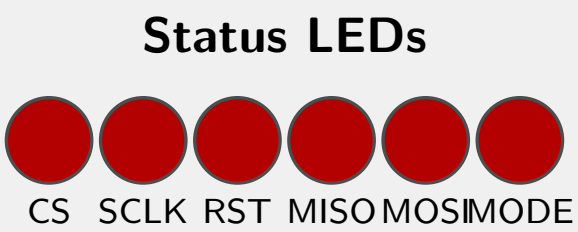
- Xilinx Artix-7 FPGA (`xc7a35tcsg324-1`) synthesized in Vivado.
- 16 physical DIP switches emulate live parallel sensor inputs (`sensor_data`).
- 8-digit multiplexed 7-segment display displays `read_reg` and `write_reg` in hexadecimal.
- Onboard status LEDs monitor CS, SCLK, MISO, MOSI, and FSM mode.



Physical FPGA Development Board Testbench.

FPGA Top-Level Wrapper Layout (`fpga_top.v`)

Artix-7 Development Board I/O Mapping



Dual 4-Digit 7-Segment Display
Read: 00FF — Write: A5A5

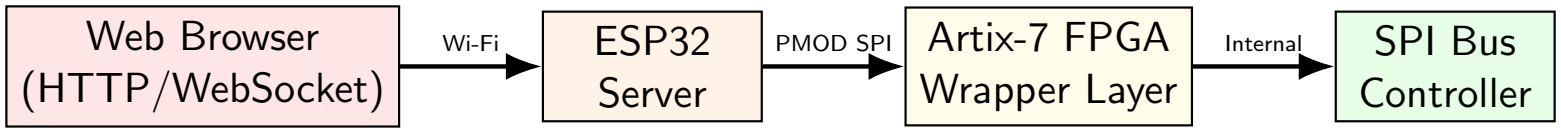


PMOD SPI

16 DIP Switches (`dip_sw[15:0]` Sensor Emulation)

ESP32 Web Server Integration Topology

- **Network Control Architecture:**
  - ESP32 microcontroller runs an HTTP / Server-Sent Events (SSE) server.
  - Provides interactive web controls (Sensor selection, Hex write payload, Read trigger).
  - ESP32 acts as SPI Master, driving hardware pins on FPGA PMOD headers.

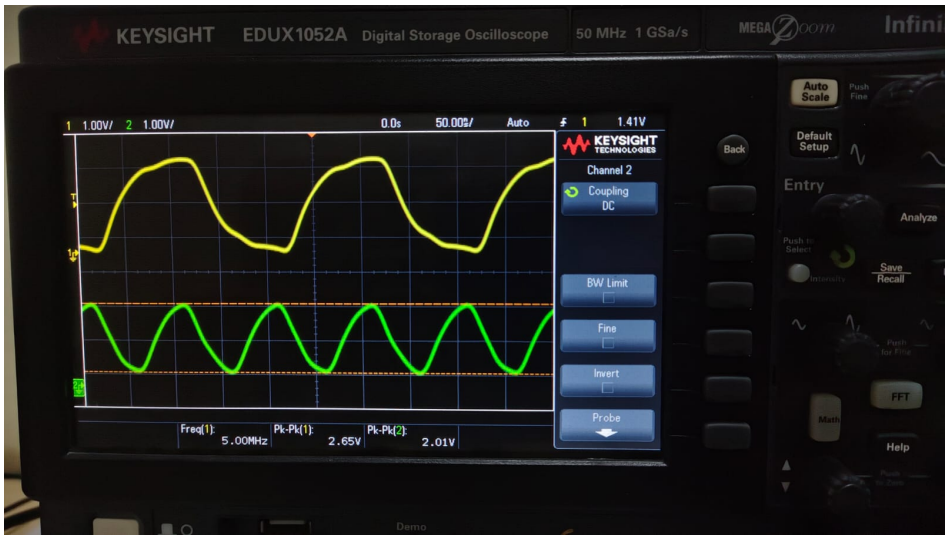


ESP32 Web Dashboard & Real-Time Telemetry



- Live status header displays `sensor_data`, `read_reg`, `write_reg`, and FSM state.
- HTML5 Canvas logic analyzer visualizes real-time SPI waveforms.
- Embedded JS simulator mirrors the 9-state `top.v` FSM for offline validation.

WHO_AM_I State Clock Frequency Tests



10 MHz Clock Capture: `WHO_AM_I` transaction shifting out `SENSOR_ID = 8'd9` on MISO.



50 kHz Clock Capture: Verification of low-speed clock stability with zero bit loss.

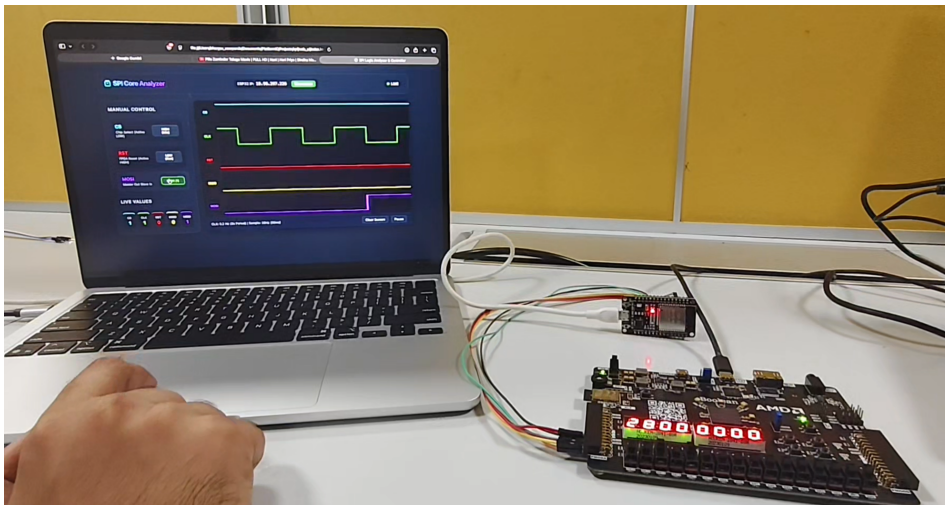
Result: Opcode `2'b11` correctly triggers `WHO_AM_I_STATE`, reliably streaming out the 8-bit identifier across wide frequency ranges (50 kHz to 10 MHz).

Hardware Sensor Data Read Capture

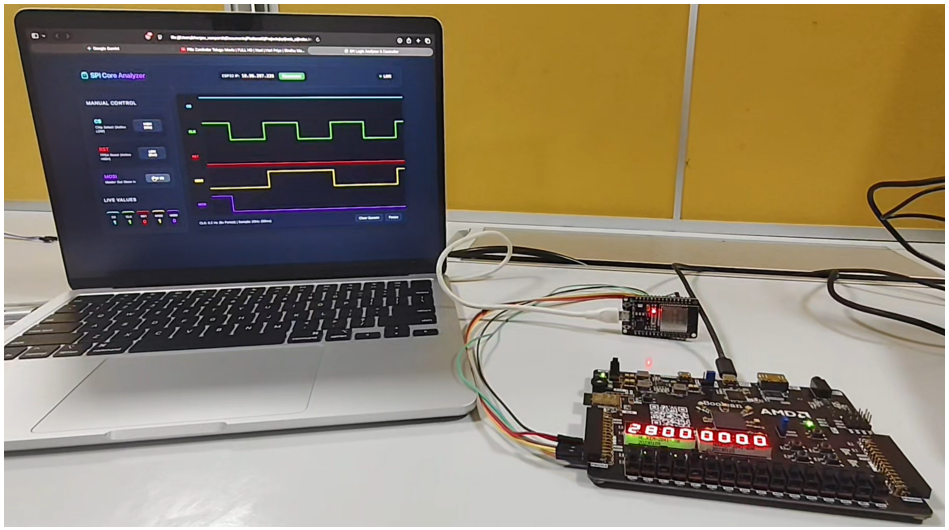


- Logic analyzer trace of Sensor Read operation at 50 kHz SCLK.
- Verifies parallel data transfer from physical DIP switches to serial stream on MISO.

MOSI Read Command & MISO Response Signals

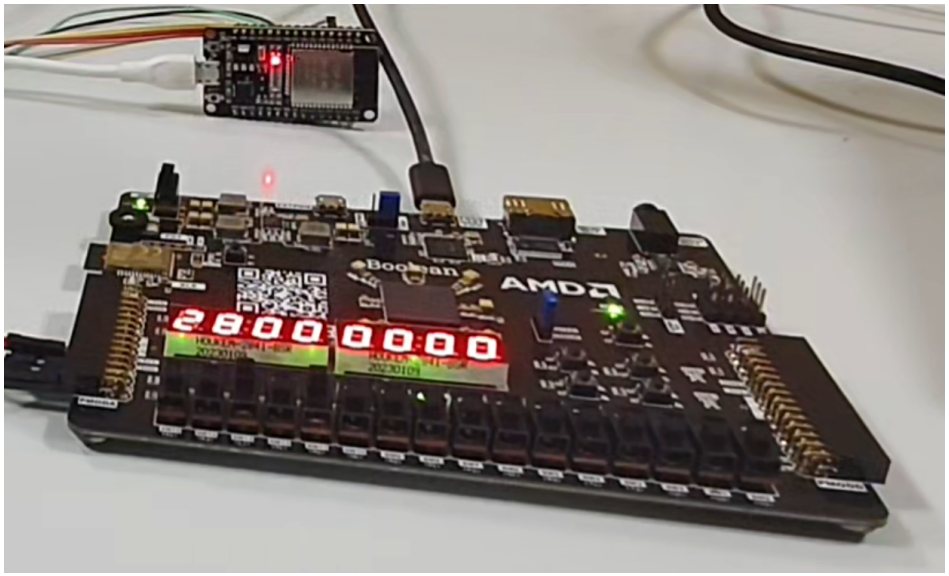


(a) MOSI Read Command: Master transmitting opcode 00 and address frame.

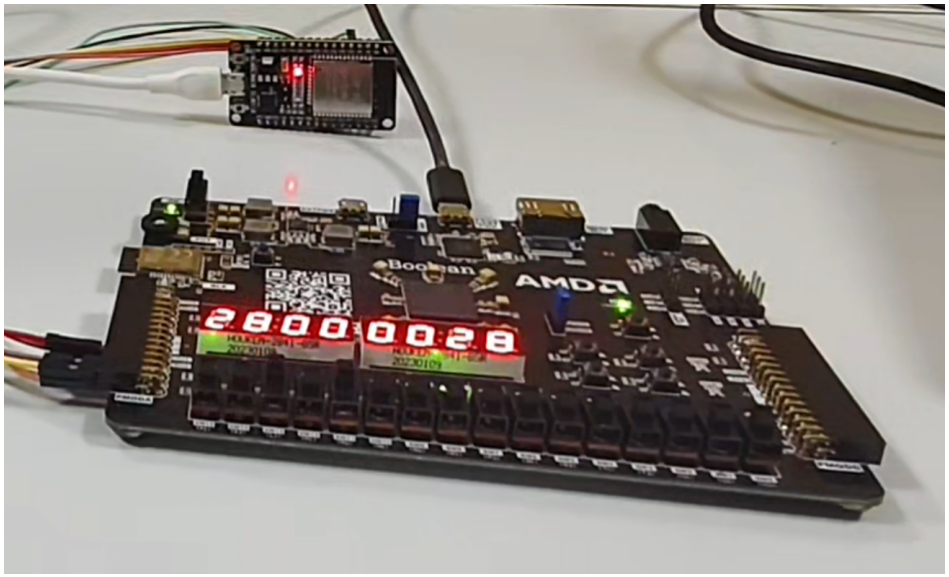


(b) MISO Read Response: FPGA streaming register data back to host master.

Write Transaction Oscilloscope Signals



(a) Write Sequence Start: Opcode 01 and address decoding phase.



(b) Write Payload Transfer: Data bits shifting into `write_reg`.

Case Study 1: Sensor Data Read (0x2800)



MISO output records 0010100000000000 (0x2800).

Cycle	MOSI	FSM State	MISO Output
1	0	DETECT	0
2	0	DETECT (Op 00)	0
3..6	1000	READ_ADDR	0
7	0	SEND_DATA	0
8	0	SEND_DATA	0
9	0	SEND_DATA	1
10	0	SEND_DATA	0
11	0	SEND_DATA	1
12..22	0...	SEND_DATA	0

Case Study 2: Register Write (0x0028)



`write_reg` successfully updated to 0x0028.

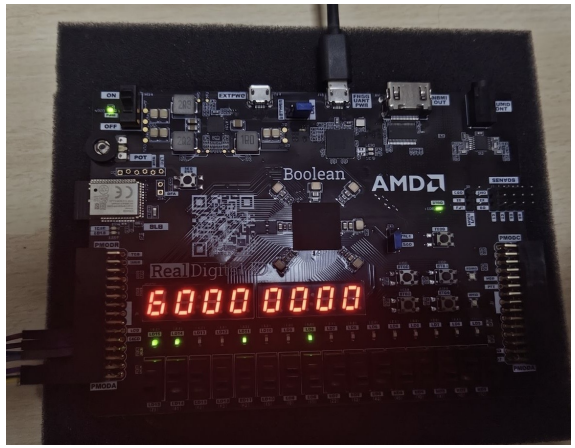
Cycle	MOSI	FSM State	write_reg
1..2	01	DETECT (Write)	0x0000
3..6	0000	WRITE_ADDR	0x0000
7..10	0101	WRITE_LEN (6b)	0x0000
11..13	000	WRITE_DATA	0x0000
14	1	WRITE_DATA (Bit 3)	0x0008
15	0	WRITE_DATA	0x0008
16	1	WRITE_DATA (Bit 5)	0x0028

Case Study 3: The 010101... Phenomenon (0xA000)

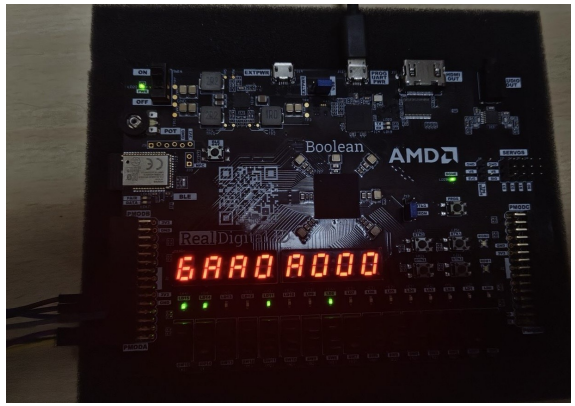
- **Observed Phenomenon:** Sending alternating bit sequence `0101010101...` turned `write_reg` from 0000 to A000.



Web simulation capture.



Initial state (0000).



Final state (A000).

Case Study 3: Step-by-Step Bit Parsing Breakdown

1. **Opcode Decoding (`DETECT`):**
   - Bit 1: `0`, Bit 2: `1` → Opcode `2'b01` (Write Command).
2. **Address Decoding (`WRITE_ADDR`):**
   - Bits 3–6: `0`, `1`, `0`, `1` → Address `4'b0101` = `4'd5` (Sensor 4).
3. **ROM Lookup:**
   - For `4'd5`, ROM yields `in_width = 4`, `in_start = 12`.
4. **Payload Writing (`WRITE_DATA`):**
   - Bits 7–10: `0`, `1`, `0`, `1` → written to bits 12..15 as `4'b1010` (`0xA`).
5. **Final Register Value:**
   - `16'b1010_0000_0000_0000` → `16'hA000`.

FPGA Resource Utilization & Timing Analysis

FPGA Resource	Method 1	Method 2	Available (Artix-7)
LUTs	78	62	20,800
Flip-Flops	68	54	41,600
BRAM Blocks	0	0	50
Setup Slack	+15.24 ns	+15.24 ns	Target: 50 MHz
Max Frequency (F_{max})	142.8 MHz	166.4 MHz	—

Table: Resource and Timing Analysis Report

- **Logic Savings:** Method 2 achieves a **20.5% reduction in LUT usage** and a **20.6% reduction in FFs** due to a streamlined state machine (6 vs 9 states).

Quantitative Protocol Latency & Throughput

Latency Equations

$$\text{Latency}_{\text{Method 1}} = T_{\text{detect}} + T_{\text{addr}} + T_{\text{len}} + T_{\text{payload}} = 11 + N \text{ cycles}$$

$$\text{Latency}_{\text{Method 2}} = T_{\text{detect}} + T_{\text{addr}} + T_{\text{payload}} = 6 + N \text{ cycles}$$

- For a standard 4-bit write transaction ($N = 4$):
  - **Method 1:** 15 clock cycles (1.50 μs at 10 MHz).
  - **Method 2:** 10 clock cycles (1.00 μs at 10 MHz).
  - **Speedup: 33.3% reduction in transaction latency**.
- **Continuous Throughput:** Method 2 reaches 7.14 Mbps vs. 5.26 Mbps for Method 1.

Methodology Tradeoffs & Selection Guidelines

Method	Advantages (Pros)	Drawbacks (Cons)
Method 1: Address + Length	- Software-defined dynamic widths - No re-synthesis needed for new sensors - High runtime flexibility	- High control overhead (11 bits/write) - Higher transaction latency - Larger FSM logic footprint
Method 2: ROM Lookup	- Low control overhead (6 bits/write) - 33.3% faster data transfers - 20.5% smaller FPGA logic footprint	- Hardcoded register metadata - Requires FPGA bitstream rebuild to update - Constrained by ROM depth

Conclusion & Summary of Accomplishments

- **Wiring Reduction:** Resolved the $O(N)$ physical wiring problem of traditional multi-slave SPI networks, reducing connections to a constant 3/4-wire bus.
- **CDC FIFO Integration:** Robustly decoupled master clock phases from sensor domains using an asynchronous Gray-code FIFO.
- **Methodology Verification:** Proven 33.3% latency reduction and 20.5% logic saving using the ROM Lookup Table protocol.
- **Complete Hardware Verification:** Fully validated via oscilloscope signal captures, Vivado synthesis on Artix-7, and an interactive ESP32 web server interface.

Future Work & System Extensions

- **Programmable RAM Configuration Map:** Replace the static ROM with write-programmable RAM, combining Method 1's runtime flexibility with Method 2's low protocol overhead.
- **Direct Memory Access (DMA):** Integrate a DMA controller to burst sensor data directly to system memory without host CPU intervention.
- **Standard SoC Interconnect Wrappers:** Develop AMBA AXI4-Lite or Wishbone bus wrappers for seamless integration into larger SoC platforms.

Thank You!

Questions & Discussion